

Exploring Stealthy Spatiotemporal Attacks in Extended Reality

Anonymous submission

ABSTRACT

In recent years, Extended Reality (XR) has metamorphosed into an essential technology for critical applications such as surgery, therapy, etc, marking a significant evolution from its early use mainly for leisure activities. This change has necessitated a more focused look at the security of XR devices and applications. Prior work on the security of XR has mainly focused on generic security aspects such as vulnerability analysis, secure authentication and device pairing, and securing private information in multi-user settings. However, due to their multi-modal and spatiotemporal nature, XR systems are also prone to attacks on sensor modalities across multiple dimensions, which has been ignored by prior work.

In this paper, we establish the importance of spatiotemporal security in XR systems. Our key insight is that concurrent and context-aware spatiotemporal manipulations on multiple sensor streams enable simple, precise, and stealthy attacks with real-world consequences. This motivates us to present a taxonomy of multi-modal (inertial, visual) and multi-dimensional (spatial, temporal) attacks in XR systems, and devise a simple and fast algorithm that decides on the right time and sensor to launch the attack. By taking the use case of a user in an office building, we demonstrate precise control over user's trajectory in a stealthy manner even in presence of state-of-the-art sensor fusion algorithm and system checks. The practicality and stealthiness of our approach makes remediation a challenging task. We finally discuss the broad applicability of our attack surface to other tracking-based applications.

1 INTRODUCTION

The immersiveness of Extended Reality (XR) applications relies on multi-modal sensing of user activities and surrounding environment via commodity sensing devices such as Head Mounted Displays (HMD), hand-held controllers, and other devices [4, 22]. XR systems are centered around tracking humans and enhancing their experience by designing human-in-the-loop systems. A key challenge most human-in-the-loop systems face is exploitation of security vulnerabilities that directly impact human safety [3]. Due to the prevalence of sensors, reduced field of view, and integration of critical applications, XR devices have become a lucrative target for malicious activities with real-world consequences to human safety [31]. Recent security trends in XR mostly focus on security and privacy issues around authentication [48, 55], access control [46], and digital biomarkers [52, 71], however, we argue that there exists other attack surfaces that are prevalent and pose harm to users in a stealthy manner.

Macro and micro-level tracking is finding its way in our everyday lives through emerging technologies in XR, autonomous vehicles and robotics. As such, tracking algorithms rely on multi-dimensional spatiotemporal relationships among multi-modal sensing streams. For instance, combination of visual cameras and inertial sensors use Visual-Inertial Odometry (VIO) for tracking in XR. Recent works have demonstrated attacks on individual sensing modalities such as inertial sensor [59] and visual sensor [9].

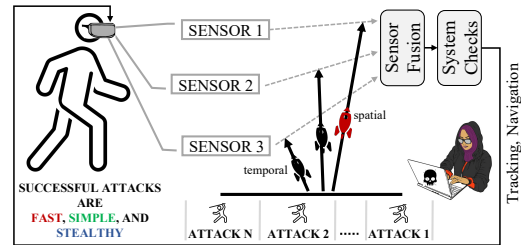


Figure 1: Extended Reality (XR) systems leverage multiple sensors for tracking and navigation applications. An attack on individual sensor can be detected and mitigated through sensor fusion and system checks, whereas, our fast and concurrent attacks on multiple sensors can be stealthy.

We show in our preliminary study that these individual sensor attacks can be mitigated by deep learning-based sensor fusion techniques such as SelectFusion [11]. In principle, when data from one sensor stream is degraded, other streams dominate the tracking algorithms for correct operation. In contrast, in this paper, we present a new attack surface that is resilient to state-of-the-art sensor fusion techniques by concurrently manipulating multiple sensing streams across multiple dimensions.

Unlike attacks on a single sensor [9, 59] that can be remediated by sophisticated deep learning techniques, our idea of launching concurrent attacks on multi-modal sensors along spatiotemporal axes demonstrate precise control over the attack and makes it hard for the system to detect abnormal trajectory. Figure 1 shows our tracking and navigation use case, where a user is wearing HMD and experiences XR. An adversary attacks multiple sensors on HMD concurrently – some along spatial and some along temporal axes – such that it deceives the sensor fusion algorithm and bypasses system checks, and successfully divert the user from its trajectory in a stealthy manner.

XR is an interactive system with human-in-the-loop and have tight time constraints. Any visual lags or glitches can indicate a user of a potential manipulation. In an attempt to launch fast and stealthy attacks on XR, we encounter two challenges: 1) How and when to launch concurrent attacks such that the malicious outcome is hidden both from the user and system checks. 2) How can an adversary leverage contextual information in a scene to control the duration and impact of attack. Our proposed design uses a variety of frame manipulation techniques across temporal and spatial axes, and craft stealthy attacks that cause slow down, speed up, zero displacement and path deviation effects on user tracking. An important design decision is to choose a metric that determines the right time and the right frame to launch attack such as similarity index in consecutive data frames. We use perceptual hashing for visual frames and locally sensitive hashing for inertial frames as a measure of similarity between adjacent frames. To precisely control the outcome, we parameterize the attacks to enable a context-aware launch, a stealthy continuation and a graceful end.

The novelty of our attack surface isn't in crafting new frame manipulation techniques, it is rather in concurrently attacking sensor streams only across spatiotemporal dimensions at the right time and for the exact duration to remain stealthy and achieve the desired outcome. We evaluate our attack surface for configurability, preciseness, efficacy, and stealthiness in a campus navigation use case. The attacks forced the user to follow a malicious trajectory at an unintended speed without the user realizing the unsafe practices and circumstances. In XR, user may experience peripheral impairment due to limited field of view, which reduces the depth of focus, and affects human distance and speed perception, and reaction time [43, 47, 56]. This effect further exacerbates the impact of our attacks on the user. Our evaluations demonstrate that our Path Deviation attack can divert the user from its trajectory, taking her upto 40meters away from the original destination without user experiencing any abrupt changes.

1.1 Contributions

Our hypothesis is that a concurrent and context-aware spatiotemporal attack on multiple sensors streams enables simple, precise, and stealthy outcomes. In evaluating our hypothesis, we make the following contributions:

Concept of Spatiotemporal Security: We introduce the concept of spatiotemporal security in XR systems. While previous work has focused on generic security mechanisms, such as authentication, to the best of our knowledge, no prior work has explored the spatiotemporal attack surface in XR systems.

Taxonomy of Spatiotemporal Attacks: We present a taxonomy of multi sensor (inertial, visual, depth) and multi-dimensional (spatial and temporal) attacks. We outline the advantages, drawbacks, and challenges of attacking the aforementioned sensors and dimensions. This knowledge informs how we devise our attacks.

Design of Stealthy Spatiotemporal Attacks: We devise *precise*, *concurrent*, *stealthy*, and *context-aware* attacks. Our attacks leverage the context and attack multiple sensors in a concurrent fashion in spatial and temporal domains. We parameterize our attacks to provide a highly precise control of user's trajectory to the attacker. Our tracking system uses a state-of-the-art sensor fusion algorithm and the success of our attacks against this sensor fusion algorithm demonstrates the stealthiness of our approach [11]. We also enhance the stealthiness of our attacks by making them simple and fast. This is necessary since XR systems sample sensor data very quickly, allowing minimal time for covert operations.

Evaluating Preciseness and Stealthiness: We extensively evaluate our proposed attacks to demonstrate their preciseness and stealthiness. By taking the use case of a user in an office building, we demonstrate precise control over user's trajectory across all our attacks in a stealthy manner even in presence of state-of-the-art sensor fusion algorithm and system checks.

Generalization of Attacks: We discuss how our attacks are generic i.e. they are applicable to any system that uses inertial and visual sensors to do tracking. The examples of these systems include autonomous vehicles, augmented reality, and robotics. Also, our attacks are applicable to any system that uses a different set and/or number of sensors to track object's/user's movement in space and time, i.e. GPS, LIDAR, infrared etc.

2 PROBLEM STATEMENT

In this work, we make a case for *concurrent, context-aware attacks* in Extended Reality (XR). We demonstrate that while an attack on a single sensor can divert a user from its path, a concurrent attack on multiple sensors enables a more precise control over the attack, making it hard for the system to detect abnormal trajectory. We show that an attacker can adversely impact critical applications by manipulating the timing and spatial properties of devices and data. Before we define our problem statement, we define relevant concepts of *Attack Surface*, *Attack Knobs*, and *Attack Vectors*.

Attack Surface: An attack surface is the collection of different points (attack vectors) where an unauthorized often malicious user (*attacker*) can try to enter data to or extract data from an environment. In this paper, we define the tracking of HMD in XR as the attack surface. To be specific, we are concerned with sensor output data and processing of that data in sensor fusion algorithm to produce the tracking in the XR.

Attack Vectors: An attack vector is the path that a attacker takes to exploit the vulnerabilities in a particular attack surface. We define a sensor modality as the attack vector if it poses the property of generating data that have spatial and/or temporal feature (attack knobs). To be precise, we demonstrate our proposed attack surface on inertial sensor and camera (visual). Covering other attack vectos, e.g., LIDAR, RADAR, IR sensor, is out of the scope for this paper.

Attack Knobs: An attack knob is defined as the dimension of an attack vector. For this work, we consider both spatial and temporal dimensions as our attack knobs. Attack knobs are strongly tied to a particular attack vector. As noted in [11], certain modalities (e.g., vision and inertial) may dominate others depending on the conditions. For example, an attack on inertial vector contributes more in presence of fast rotation, which indicates that it has strong temporal component attached to it. On the other hand, visual vector features are preferred during large translations and have strong spatial component attached to it. While designing our attacks, we use this complementary nature in dimensionality to decide the suitable attack knobs for our attack vectors.

Given the aforementioned definitions, we next discuss our problem statement that outlines the challenges in launching *concurrent context-aware attacks*. Our hypothesis is that *a concurrent attack that uses appropriate attack knobs on multiple attack vectors enables simple, precise, and stealthy attacks*. We outline various research challenges in evaluating our problem statement below.

- (1) How to coordinate sensor attacks, as mentioned in [9, 59, 62], across multiple attack vectors with different knobs for each vector in a resource- and time-constrained environment such as XR? This challenge motivates *simplicity*.
- (2) What should the nature of the attack be when it is launched? Should it be slow and long, fast and short, or random? These questions are answered to ensure *stealthiness*.
- (3) How to determine the best time to launch an attack on a single vector and then correlate it with other vectors with different knobs? This challenge motivates *preciseness*.

Our proposed attacks rely on **simple** frame manipulations that are not time- or compute-intensive (§3.2). As a results, they can be launched quickly even in a resource-constrained environment of XR. To ensure **stealthiness**, we devise an algorithm that identifies the best time to launch and continue the attack (§4). The proposed

algorithm also parameterizes the attack, allowing the attacker to launch a **precise** attacks that results in a desired outcome (§4.2).

3 ATTACK SURFACE SETUP AND OVERVIEW

In this section, we provide an overview of our setup. In doing so, we discuss our threat model, use case, and a preliminary study.

3.1 Threat Model

We assume a scenario where an attacker is attempting to gain the control of the tracking pipeline in HMD that is driven by sensor output and sensor fusion.

Attack Scope. We assume that the attacker can neither directly access the digitized sensor readings nor physically touch the sensors in Extended Reality (XR) headset. She can only exploit vulnerabilities when the data is being generated on the sensor level and has not been moved to any of the data processing pipelines (e.g., tracking, sensor fusion).

Sensor Access. In case of inertial sensor, the attacker can gain access by emitting acoustics to affect the integrity of sensor data, by manipulating analog signals on the signal conditioning path before the data is digitized or processed [59]. For visual data (camera), we assume that she has compromised a network link between the camera and XR headset data processing pipeline and/or the authentication material like secret keys used by the camera in headset [64]. We assume the attacker can access the communication link between the system verifier and the headset and has the ability to buffer significant amount of visual frames [62].

Camouflage. The attacker will essentially use two types of alteration in the frames: spatial and temporal. All the alteration when combined across different sensors can look like small error which can be ignored by the tracking algorithm. Like spatial alteration can be perceived as misalignment that can occur due to axis misalignment and the incorrect sensor calibration [34]. The time shift between windows of input images and windows of inertial measurements can be perceived as a relative drifts in clocks between independent sensor subsystems [37].

Proximity. The attackers do not have access to the headset itself or the firmware but attacker is in close proximity of the headset. However, the attacker can not touch the device physically [59, 60].

Context Awareness. Since the attacker assumed to be in close proximity, attacker knows the context of the scene. For example, attacker can launch *attack A* if user is walking slowly in the indoor space with close hallways, and could launch *attack B* if user is walking slowly in indoor space with large room [44, 53, 66].

Device Awareness. While the attacker has no access to the device physically, she possess the means to get a hold of similar device to learn about the behaviour of sensor fusion algorithm, system verifiers, frame rate of sensors, architecture, and parameters.

3.2 Types of Frame Manipulation

As discussed in § 2, our attacker is limited to using only basic frame manipulation techniques. The type of frame manipulations outlined in this section are not novel. Our contribution lies in leveraging these simple manipulations to launch a wide-range of attacks that we describe in § 4. We next describe the different types of manipulations that we use in our attacks.

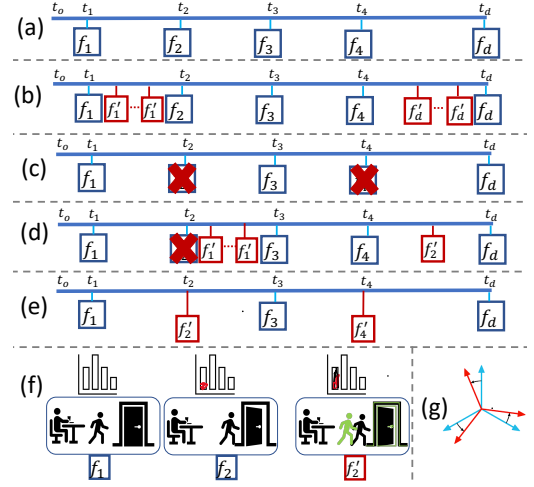


Figure 2: Various frame manipulation techniques. (a) original, (b) stretch, (c) compress, (d) shuffle, (e) misalign, (f) histogram shift, and (g) orientation shift.

Stretch. With this manipulation approach, attacker constructs falsely long paths by adding more frames between two consecutive frames [54]. It increases the path length between *source* and *destination*. This happens due to feedback loop between sensor data and user application (App). The App continuously tracks user location via HMD and updates user's display accordingly. Upon reacting to display visuals, user's location is changed which is further tracked by the App. At a given moment t_1 , if the attacker adds multiple copies of f_1 between consecutive frames f_1 and f_2 , tracking algorithm gives a wrong illusion that user is standing near the old position. The App sends this navigation feedback to the user and adds more steps between the source and destination path as seen in Figure 2(a). This results in dilation of time from t_1 to t_2 , leading to the slow down effect. The attack-induced malicious path diverges from the original path, as shown in Figure 2(b).

Compress. In this manipulation, the attacker drops frames to reduce the steps that user is taking from the *source* to *destination* [23]. As shown in Figure 2(c), as the attacker drops the frames, the sequence of frames scene by the tracking algorithm is changed. For example, if the attacker drop f_2 and f_3 , f_4 will become new f'_2 and so on. The tracking algorithm perceives that the user is walking faster than the actual speed. This is because the algorithm sees a more drastic change in the frame characteristics between adjacent frames. The App will perceive that the next point is closer, decreasing the overall distance between two points. It will make the user walk in a more straighter trajectory between source and destination.

Shuffle. This approach manipulates the frames both temporally and spatially. In a temporal manipulation, the attacker can switch between adding and dropping frames, as shown in Figure 2(d). For example, attacker can drop f_2 at its original time and buffer it to use it in place of f_6 . It may also add some forgery in the frames to corroborate with the current tracking scenario. The goal with this type of manipulation is to diverge the user from its intended path by using a combination of speed ups and slow downs.

Misalign. In this approach, attacker manipulates frames to deviate the user from its original path from *source* to *destination* while changing the destination. Tracking algorithm looks at the f_i to f_{i+m}

frames to determine the position and orientation of the user, where m is the window size of history for the tracking algorithm [50]. If the frames between i and $i + m$ are demonstrating a certain pose (combination of position and orientation vector) then the spatial changes in the frames f'_i to f'_{i+m} can be used to manipulate the pose. As shown in Figure 2(e), attacker adds f'_2 in place of f_2 which is spatially manipulated version of f_1 and f_2 . The end goal of this manipulation is to spatially modify inertial and visual frames to make tracking algorithm get the wrong pose, and deviate the user to altered destination instead of the original destination.

Histogram Shift. Histogram of an image shows the density distribution of pixel intensities and provides information about the objects in a given image. In this approach, attacker manipulates an image such that the difference in histogram between adjacent frames does not exceed a threshold, whose value is determined empirically. One method to launch this attack is where the attacker takes a difference between the current frame i and the previous frame $i - 1$. She then uses edge detection to identify the segments of the image that are different from the previous frame. Finally, she applies this "difference image" to the new image such that the histogram difference does not exceed the aforementioned threshold value. The goal of finding and respecting the right threshold is to avoid making drastic changes and remain stealthy. This allows the attacker to manipulate the spatial map of the surrounding leading to inaccurate positioning and tracking, as shown in Figure 2(f).

Orientation Shift. The attacker spatially modifies the inertial data to change the pose information in a frame. Pose information can be found from inertial data. The attacker will detect the pose from the inertial frame and change the orientation to opposite of what frame is currently reporting as shown in Figure 2(g). This will cause tracking algorithm to detect false pose data and forwarding wrong signal to applications causing false navigation and interaction.

3.3 Use Case: Indoor Navigation and Tracking

To evaluate our attacks, we take the indoor positioning and tracking of humans as a use case. We consider two concrete applications in this scenario. In the first application, a person is trying to navigate an unfamiliar building such as college, museum, office, warehouses, and escape rooms while avoiding dependency on other human beings [2, 20, 69]. Navigating in unknown places can be challenging for people in an unfamiliar locality. Recent advancements in navigation systems focus on helping users understand the structure of their surroundings while navigating unfamiliar spaces. XR localization and tracking can guide user by informing about the current location and the path to follow, making it possible for user to move in a building without any human assistance. It can reduce the cognitive effort required in navigating the unfamiliar and/or complex spaces and boost engagement through multisensory interaction.

In the second application, we consider a person who is using an indoor running/jogging track inside a gymnasium building. XR application is helpful in guiding the runner to have an effective run by maintaining a desired and safe jogging/running pace. Additionally, XR indoor navigation and tracking can also help in running and maintaining the facilities from remote [17]. Along with that more complex and precise application lie in medical facilities, where surgeons can use XR in surgery and tracking can help tie medical imagery on top of patient for precise incisions and cuts [65].

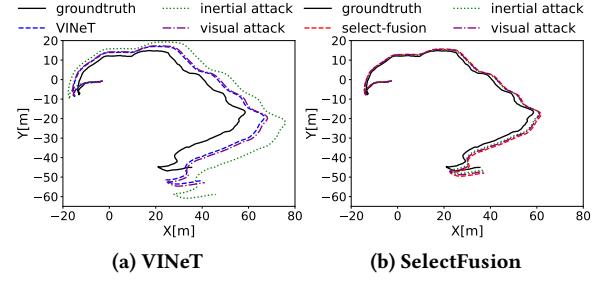


Figure 3: State-of-the-art sensor fusion algorithm, SelectFusion [11], is effective against attacks on a single sensing stream whereas simple tracking algorithm, VINet [13], fails.

Our attack surface can pose significant danger in aforementioned situations. The dangers range from less harmful case of navigating to a wrong room or destination to hitting a wall resulting in a physical damage. Similarly, the attacker can cause a variety of problems for an indoor jogger/runner, which range from the jogger/runner having a less-effective run/jog (unevenly paced, non-ideal speed) to physically hitting a person or an object at high speed. Similar dangerous situations can be envisioned for more critical indoor applications of surgery, construction, and maintenance.

3.4 Preliminary Study

The purpose of this study is to demonstrate that state-of-the-art data processing techniques are capable of detecting and correcting for manipulations on single attack vectors (inertial or visual). We show how a simple tracking algorithm, VINet [13], fails to detect the attack, while a state-of-the-art sensor fusion approach, SelectFusion [11], is able to detect the attack and make appropriate corrections to mitigate the effect. We launched attacks that aims to increase the distance travelled by the user, discussed in § 4.2.2, on our inertial and visual streams individually. We used temporal and spatial attack knobs for inertial and visual attack vectors, respectively. Figure 3(a) shows the ground-truth 2D trajectory, trajectory estimated by VINet, trajectory after the inertial, and trajectory after the visual attack (mean squared error 0.167 and 0.0842 respectively). We can see that VINet is not able to detect the attack and mitigate its effect. Similarly, Figure 3(b) shows the ground-truth trajectory, trajectory estimated by SelectFusion, and trajectories after the inertial and visual attacks (mean squared error 0.0208 and 0.0169 respectively). We can see that SelectFusion is not only better at estimating the user trajectory, but is also able to detect and mitigate the attack effect.

SelectFusion can detect an attack on an individual vector by comparing it against the other available vectors. We argue that the attacker can coordinate manipulations on multiple vectors and overcome detection and mitigation mechanisms of such sensor fusion techniques. This insight motivates our concurrent attacks on multiple vectors to achieve successful and stealthy attacks.

4 STEALTHY SPATIOTEMPORAL ATTACKS

In this section, we present multiple concurrent spatiotemporal attacks that use simple frame manipulations discussed in § 3.2. We also discuss the research challenges that arise in a successful, context-aware launch of the outlined attacks. While exploring the research challenges, our attack approach (illustrated in Figure 4) takes in the sensor streams of different attack vectors, uses the

Attack	Frame Manipulation	Effect	Attack Vector	Attack Knob
Slowdown	stretch	distance enlargement	camera	temporal
			inertial	temporal
Speedup	compress	distance reduction	camera	temporal
			inertial	temporal
Zero Displacement	stretch, compress, shuffle, histogram shift	route change	camera	temporal, spatial
			inertial	temporal
Path Deviation	histogram shift misalign	destination loss	camera	spatial
			inertial	spatial

Table 1: Summary of Spatiotemporal Attacks on XR Navigation

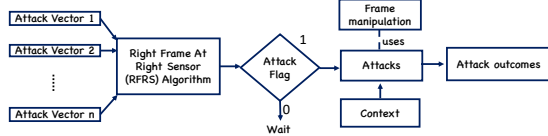


Figure 4: Our Approach to Launching Context-aware and Stealthy Spatiotemporal Attacks.

Right Frame at the Right Sensor (RFRS) algorithm to decide when to attack, picks an attack based on the context, and leverages simple frame manipulations to launch the attack. We next describe each of the design component in detail.

4.1 Proposed Attacks

We developed four attacks to demonstrate the validity of our attack surface. The goal of the attacker is to shift user's "reality" by leveraging concurrent and context-aware manipulation of atleast two attack vectors. The choice of manipulations depends on the context. We next describe the proposed attacks (summarized in Table 1).

4.1.1 Slowdown Attack. In this attack, an attacker uses the *stretch* frame manipulation technique to add frames in the sequence and give the application an impression of user slowing down. The attacker uses temporal knobs to coordinate the manipulation of inertial and visual vectors. To do so at time i , she can decide to hold the last frame that arrived at time $i - 1$. Alternatively, as she has a buffer capacity, she can replay a sequence of frames from the history. In our design, we use the holding approach as it is simpler and faster to implement in practice. The end effect of this attack is that the user diverts from the original trajectory and the application perceives that the user has slowed down. It must be noted that this same attacking approach is concurrently launched against both the inertial and visual streams.

This result makes it an ideal attack when a user is using an Extended Reality (XR)-based indoor tracking applications, similar to the application presented in the prior work for campus navigation [38]. For instance, a user wants to reach from a *source* to a *destination* point in a given time. This attack takes the user to the final destination, albeit through a longer path than intended. In another scenario, the attacker can use slowdown effect on a user trying to jog at a fixed pace. This slowdown effect will deceive the application to estimate user jogging slowly, and respond by asking the user to run faster leading to harmful consequences [27].

4.1.2 Speedup Attack. The goal of this attack is opposite of the slowdown attack. The attacker will use the *compress* frame manipulation technique to concurrently drop frames from both the inertial and visual sensing streams. It will give the application an impression of user speeding up. The attacker uses the same temporal knobs to manipulate the inertial and visual attack vectors exposed

by our attack surface. This attack is also suited for the XR-based indoor navigation and tracking application that we discussed in the context of the previous attack. In the first scenario from the previous attack, this attack takes the user from the *source* to the *destination* in a straighter trajectory, which may be harmful due to the presence of other objects and humans in the trajectory. In the second scenario, the speedup illusion caused by the attack may result in the application asking the user to jog at a slower pace, leading to an ineffective jogging session.

4.1.3 Zero Displacement Attack. Just like the Slowdown and Speedup attacks, this attack also takes the user from the *source* to the *destination* resulting in a zero displacement for the overall trajectory. However, this attacks uses a combination of *stretch*, *compress*, *shuffle*, and *histogram shift* manipulations to make sure that the distance travelled by the user is also the same as the original trajectory. Unlike the previous two attacks that just used temporal knobs, this attacks also exercises the spatial knob using histogram shift to mask the differences in the frames when they are shuffled. Just like previous attacks, any frame manipulation is concurrently applied to both the inertial and visual sensing streams. This attack is useful when the attacker wants the user to go through a specific point outside the trajectory or she wants the user to navigate from *source* to the *destination* at an uneven pace.

4.1.4 Path Deviation Attack. This attack uses the spatial knob to manipulate frames across both inertial and visual vectors in our attack surface. The attacker concurrently employs *misalign* and *histogram shift* frame manipulations for the inertial attack vector and the visual attack vector, respectively. The tracking application is fed the wrong pose information about the user, which results in an incorrect trajectory estimation from the application. As a result, the user trajectory will deviate from the original trajectory and user will lose the destination. Its easy to envision how this can be very dangerous for simple navigation or jogging/walking use cases.

4.2 Research Challenges

Our proposed attack strategies employ *simple* frame manipulations using various *attack knobs* to attack multiple *vectors* to achieve a desired result. However, launching these attacks in a *concurrent*, *fast*, and *context-aware* manner poses significant research challenges (RCs) that we next highlight and solve.

4.2.1 RC 1: When to attack a given vector? The discussion on frame manipulations (§ 3.2) outlined "how" to attack a given vector. Given the mechanism, the next question is "when" to launch the attack on a given vector? Our threat model poses that the attacker does not have any information about the underlying tracking algorithm, sensor fusion algorithm and/or any correcting mechanism, a knowledge that is often proprietary [29]. Therefore, we avoid

using an approach that requires extensive knowledge and/or profiling, and rely on general domain knowledge and public information about the hardware [8, 51]. One such observation is that **similarity** between frames is an important metric for determining user pose and semantic similarity for tracking and visual inertial odometry [14, 35, 45, 58, 70, 72]. With this observation, we take **similarity index** as the basic metric for deciding when to attack a given vector. The choice of a similarity index depends on the kind of vector. We next define similarity index and outline how to compute its value for various vectors in our attack surface.

Similarity Index. It is a ratio of semantic overlap between adjacent frames f_i and f_{i+1} . A value of 0 means that there is no overlap between two frames. A value of 1 means the two frames are same. To find the similarity index, we use hashing of a frame as the underlying metric and quantify the similarity between two hash keys as our similarity index. We can use any other property as the underlying metric for the similarity index as long as it quantifies the geometrical information for the given attack vector. Other similarity metric examples include key-frame based similarity detection [50, 70] and inter frame correlation [39, 40]. We use the hashing function as it is robust to minor changes in frames. Also, hashing is useful in detecting errors and flagging inappropriate data frames by comparing frame hashes with a dataset of known frame hashes [21].

Hashing for Visual Attack Vector. We use perceptual hashing to quantify similarity index for images [49]. Perceptual hashing examines the contents of an image and constructs a hash value that uniquely identifies an input image. Given the hash values for a set of frames, we compute the similarity index as the difference between the hashes based on the hamming distance [30].

Hashing for Inertial Attack Vector. We use locality sensitive hashing for the inertial frames [26, 67]. To compute the similarity index, we use cosine distance which is defined as an angle between two vectors. On any dimension, the angle between two vectors always ranges from 0° to 180°, which defines the distance. Unlike the Euclidean distance, the cosine distance is scale-insensitive [25].

We use the similarity index to decide when to attack and launch concurrent attacks across attack vectors. We discuss that as a part of the next research challenge.

4.2.2 RC 2: How to concurrently launch attacks across vectors? The data generation and update rates of sensor streams in XR systems tend to be extremely fast. For example, the sampling rate for the HoloLens2 is around 30 frame per second [24]). This necessitates two key requirements for a successful coordination of attacks across vectors. First, both the decision when to launch the attack and the actual launch of the attack must be fast and complete within the sampling window. Second, there should be a common metric that provides a reference for attacks across vectors.

We satisfy the first requirement by prioritizing "fastness" at every step of our attack pipeline. We launch attacks only at the data production level, which helps in coordination by avoiding the time spent in data processing and system checks. As pointed out earlier, our attacks leverage simple frame manipulations that are extremely fast. Our microbenchmark evaluations (§ 6.6) show that the end-to-end process of launching our attacks, while considering buffering of frames, takes less time than the sampling window (1000ms/30=33.33ms). Our use of similarity index as an indicating metric for launching attack helps us in coordinating attacks across

Algorithm 1: Right Frame at the Right Sensor (RFRS) algorithm: To decide the start and end of an attack on an incoming sensor stream.

Input: $frame, history_length, upper_bound, lower_bound, steadystate_length, attack_length$

Output: attack flag

```

1 attack flag  $\leftarrow$  0
2  $history \leftarrow$  FIFO buffer of size  $history\_length$ 
3 while frames are coming do
4    $history.append(frame)$ 
5   if history buffer is full then
6      $sim\_index \leftarrow$  similarity index of adjacent frames
7      $low \leftarrow percentile(history, lower\_bound)$ 
8      $high \leftarrow percentile(history, upper\_bound)$ 
9     if  $sim\_index$  in range( $low, high$ ) then
10       $steadystate\_count++$ 
11   else
12      $steadystate\_count \leftarrow 0$ ; attack flag  $\leftarrow$  0
13     if ( $steadystate\_count \geq steadystate\_length$ ) and
14       ( $attack\_count \leq attack\_length$ ) then
15        $attack\_count++$ ; attack flag  $\leftarrow$  1
16   else
17     attack flag  $\leftarrow$  0

```

different vectors. We compute the similarity index for individual vectors and initiate attacks when the similarity index value for the vectors under consideration are within a specified range. Tracking the similarity index values across multiple vectors is very simple and not compute- or time-intensive.

4.2.3 RC 3: How to attack multiple vectors in a context-aware, stealthy manner? As we discussed in RC 1 & 2, we use similarity index to decide on when to launch and to concurrently attack across multiple vectors. However, the similarity index can take a value between 0 and 1 depending on how similar are the adjacent frames. If the similarity index does not change over some time period, it means that the scene is not changing and vice versa. We propose that, for an attack to be stealthy, it should be launched when the similarity index has achieved a steady-state. The hypothesis is that, during the steady-state, attack will be stealthy as the small frame manipulations that the attacker will make will be interpreted as the variations in similarity index that are already part of the sensing stream. The attack should not be launched during the transient state as it may result in drastic changes in the scene that can get detected. Furthermore, the launch of attack during steady-state allows the coordination between attack vectors, e.g., attacker may launch attacks when both vectors are in steady-state (RC 2). This choice also presents another definition of context-awareness as the attacker launches attacks only in a certain context (steady-state) when the scene is not changing.

4.2.4 RC 4: How to enable a fine-grained and precise control over an attack? In discussing RC 1-3, we identified the metric that we use to launch the attack, how to concurrently attack across vectors, and how to make the attacks stealthy and context-aware. While this information is enough to launch an attack, it does not explore how an attacker can achieve a fine-grained and precise control over the results. To provide precise control over the outcome, we **parameterize the attacks to enable a context-aware launch, a stealthy continuation, and a graceful end.** Algorithm 1, Right Frame at the Right Sensor (RFRS) is our solution to the parameterization problem. We next define the key terminology.

1. history window ($history_length$): Number of frames that an attacker will take in to account for making any steady-state decision.

2. steady-state window (*steadystate_length*): steady-state window is the number of frames that need to be within a given range to label that the steady-state has been reached. We experimented with several window sizes and determined that for our scenario *steadystate_length* = 10 is suitable. If we choose a bigger window, there are less opportunities to launch attack. If we choose a smaller value, we would attack more frames at a time and may get caught.

3. steady-state bounds (*upper_bound*, *lower_bound*): An attack vector is said to be in a steady-state when its similarity index values has been in a given range for the duration of a steady-state window. This range is defined by upper and lower bounds. In our design, we set the value of upper and lower bounds as the Xth percentile (X_p) and 100- X_p of similarity index in the history window, respectively. Here, X can take a value between 51 and 100.

4. attack length (*attack_length*): Number of consecutive frames that an attacker can attack in a given segment.

Now that we have described all the key terminologies, we next describe how the RFRS algorithm works.

Right Frame at the Right Sensor (RFRS) Algorithm. Algorithm 1 shows an automated mechanism for picking the frames to attack while staying stealthy. It takes a stream of *frames* and other attack parameter (configured by attacker) as an input and outputs the *attack_flag*. If the *attack_flag* is 0, it is not safe to initiate or continue an attack. If the *attack_flag* is 1, the attacker can initiate or continue attack. To ensure coordination, we only launch attack when the *attack_flag* is 1 for all the attack vectors.

The inner mechanism of the RFRS algorithm is purposefully very simple. It simply buffers the incoming frames in a First In First Out (FIFO) buffer of size *history_length*. Once the history buffer is full, the algorithm computes the similarity index for all the frames in the buffer. After that, it finds the upper- and lower-bounds for the steadystate based on the attacker-configured parameters. If the similarity index lies in the specified range, a counter for the steady-state is updated. A steady-state is reached if the scene is consistent and the similarity index remains in the specified range for a period of *steadystate_length*. Once the steady-state is achieved, the RFRS algorithm will signal the attacker to initiate the attack. The attack will continue until the similarity index value goes outside the specified range or the number of frames attacked reach the *attack_length*. In this case, all the counters are reset and the algorithm starts searching for the next steady-state.

We demonstrate in the evaluation that simply by controlling the value of the RFRS input parameters, an attacker is able to precisely control the effect of an attack in a fine-grained, and stealthy manner.

5 EXPERIMENTAL SETUP

We describe the experimental setup for our evaluation here.

Data. In our work, we focus on human tracking for navigation purposes in two office buildings on a college campus. Users (authors of the paper) slowly walked through the buildings entering and navigating all the open common rooms and labs. Data is collected by the Hololens 2 headset [24]. The ground truth tracking is generated by the research mode [61] in Hololens2. For evaluating our attacks, we have collected data from Visible Light Cameras (VLC) at 5-30 framerate per second (fps) and raw inertial sensor data at 12-20 fps [61]. Data is ported to the local device via device portal service by Microsoft over HTTP. Every data frame has timestamp attached

to it. We have matched the timestamps of closest inertial data to the VLC data frame for sensor fusion. Overall, we collected 54032 samples (52 minutes with mean 17 fps) of data.

Tracking. Tracking is a major component in any system like Extended Reality (XR) involving visual-inertial odometry. To measure the effect on our attack surface, we are employing state of the art neural visual-inertial odometry architecture that is presented in [11], consisting of visual and inertial encoders, feature fusion, and pose regression. Our tracking module takes raw image and inertial frames then generates corresponding pose transformation. We train our model on our 54032 data samples with 75% - 25% train and test split. We adapt our implementation architecture from SelectFusion [11] to adjust to our dataset and train for only the stochastic hard fusion. We also implement VINeT [13] which is neural visual-inertial model without feature fusion to conduct our preliminary case study in § 3.4. We implement both architectures with PyTorch and trained on a NVIDIA GeForce RTX 2070 GPU with batch size of 8 using the Adam optimizer, with a learning rate $lr = 1e^{-4}$. Training and testing mean squared errors came out to be 0.0095 and 0.017 for SelectFusion and 0.016 and 0.06 for VINeT.

Metrics. In this section, we outline the metrics to evaluate the efficacy of our attacks. To evaluate how the overall system perceives the effect of our attacks over time, we use the Cumulative Density Function (CDF) of the similarity index over the attack sequence and compared it with the original sequence. This metric is commonly used in prior work to detect the attacks on the system [42]. CDF demonstrates that the system is not attacked if the overall frame distribution has not changed. This is because typical frame distributions and relationship with subsequent frames do not change drastically in a short period of time for particular scenarios.

To quantify the similarity between different distributions, we compute Kolmogorov-Smirnov test [1]; it is a general non-parametric method for comparing two sample sets. It quantifies a distance between the empirical distribution functions of two samples. It compares the results under a hypothesis that the samples are drawn from the same distribution. It tests for any violation of that hypothesis – different medians, different variances, or different distributions. It can detect changes in the shape of the distributions. Attacker needs to keep this value as small as possible as it represents how close are the distributions for the attacked and original sequence. Smaller the *ks test statistic* value, the better. We also quantify the difference between arithmetic means of both sequences to find the variation among similarity index values. Bigger the mean difference, more difference between both sequence.

6 EVALUATION

Our evaluation of the attack surface demonstrates that i) our proposed attacks are highly precise and configurable, ii) the attacks are effective and stealthy, and iii) the attacks work end-to-end to achieve a desired objective.

6.1 Configurability and Preciseness of Attacks

An important attribute of an effective attack is that it offers the attacker a configurable lever to control the attack. This lever allows the attacker to achieve a precise goal even in dynamic situations. Our RFRS algorithm (Algorithm 1) offers an attacker multiple configurable parameters that determine how many frames will be attacked

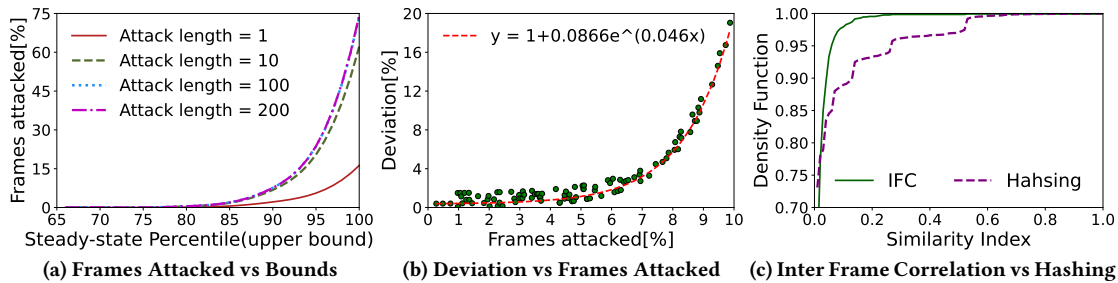


Figure 5: Configurability and Preciseness of RFRS algorithm (Algorithm 1).

over the length of the attack. By choosing these parameters, the attacker is making a local decision that will have an impact on the overall attack. We perform a set of experiments that quantify these relationships and help attacker determine how to launch the attack for a particular objective. Due to space constraints, we only show the effect of *attack_length*, as it impacts the deviation the most.

Figure 5a quantifies the relationship between steady-state bounds and the steady-state length parameters. To reiterate, steady-state bounds define the acceptable range for the similarity index values. Steady-state length controls how long the similarity index needs to be in range for a sequence to be labelled in steady-state. We can see that if the bounds are set too low, there are not enough sequence of frames that can be attacked. There is a negligible number of frames to attack if the steady-state bounds are set to a lower value than 80% (upper bound) and 20% (lower bound). The number of frames that can be attacked increases significantly at 90% upper-bound value and encompasses almost all the frames if the bound is set to 100%. The different curves show the effect of the steady-state length on the frames attacked. While the frames attacked increase from 1 to 10, but there is only a marginal increases beyond that.

Figure 5a just shows the impact of configurable parameters on the total number of frames attacked. It does not inform the attacker on how much deviation will be achieved if a particular number of frames are attacked. To do that analysis, we varied the number of frames attacked (by carefully configuring the attack parameters) and measured the deviation achieved under a wide-range of office scenarios. Figure 5b shows the actual data points and the fitted exponential curve to the data. We can see that the relationship (shown in the legend) is not linear; deviation increases exponentially with the number of frames attacked.

The exponential relationships exhibited by both the parameters and the frames attacked dictates that the attacker would need to be really careful in devising the attacks. Furthermore, this relationship was determined empirically for a given setting of an office building. A non-drastic change in an office setting will not affect this relationship. If an attacker has an access to this information, they do not need to construct their own empirical analysis. However, the same relationship may not hold true for a different setting, e.g., outdoors, and the attacker would either need to gather data to figure out the relationship or get this equation from a third-party attacker. Figuring out this relationship is not hard as it remains the same for a given setting and Extended Reality (XR) device.

The results illustrate that the attacker has a trade-off between risk and reward. If an attacker wants to achieve a large deviation, she has to attack a larger %age of the frames which may open the attack to detection. On the other hand, to avoid detection, the attacker needs

to attack less often but it may not achieve the desired deviation. This situation is further complicated as an increase or decrease in some of the attack parameters leads to significant changes. For example, the achieved deviation would become almost zero below a steady-state bound of 80% and becomes too large above 95% steady-state bound. As a result, the attacker should remain in a range that offers a more steady relationship between the attack parameters and the deviation. In our use case of office building, this range lies between 85%-95% steady-state bound and the attack length of 1. Within this range, the attacker attacks only 10% of the frames and achieves a deviation in the range of 2-20%. While the attacker can stay in this range using any combination of the attack parameters, we use *attack_length* as the primary configurable parameter to simplify attack design and evaluation.

We also wanted to explore how the use of various similarity metrics works for our given scenario with attack vectors like camera and inertial. We compared the metric of our choice, hashing, with another well known similarity metric called Inter Frame Correlation (IFC). IFC measures the correlation among consecutive frames. Figure 5c shows the Cumulative Density Function (CDF) that is calculated on a single run of our algorithm. We can see that when IFC is unable to detect the different nuances of frames similarity index. Hashing captures a more nuanced picture of the similarity index for our given scenario.

6.2 Efficacy and Stealthiness of Attacks

In this section, we evaluate our attacks to demonstrate their effectiveness, preciseness of control, and stealthiness.

6.2.1 SpeedUp Attack. In launching a *SpeedUp* attack, the goal of the attacker is to give the application an impression that the user is walking at a faster speed than its actual speed. The end effect on the user will depend on the goals set for the application by the user. If the user's objective is to try to walk at a desired fixed speed, let's call it X , from a point A to B . The expected result from the attacker is that the application will generate a feedback to slowdown the user, leading to an actual slowdown effect for the user.

Figure 6 shows the effect of *Speedup* attack under varying attack parameters and its effect on the similarity index distribution. To reiterate, an *SpeedUp* attack is achieved by dropping frames causing the scene to change more rapidly. The tracking algorithm, Select-Fusion in this case, does not perceive this as a gap in time, but instead the user moving faster. Figure 6a shows that the speedup after the attack is small. However, the attacked trajectory has less number of frames as some of the frames were dropped. The change in speed and the difference in the number of frames is more visible in Figure 6b, where the attacker launches a more severe attack with

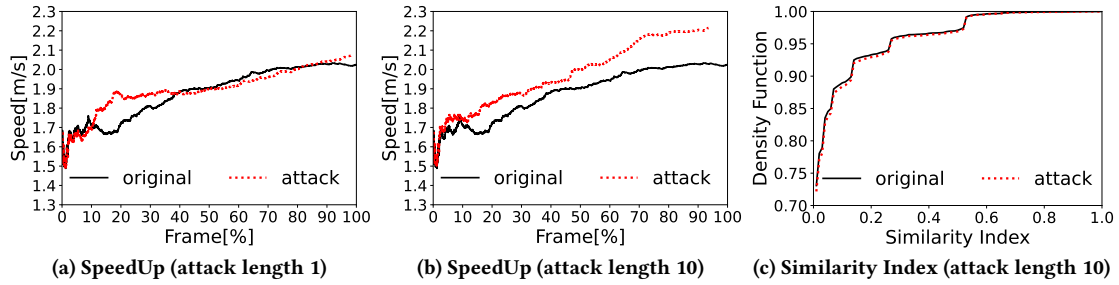


Figure 6: SpeedUp Attack. The magnitude of SpeedUp depends on attack length (small at 1 (a) vs large at 10 (b)). (c) shows that the distribution of hashing function which does not change before and after the attack, demonstrating stealthiness.

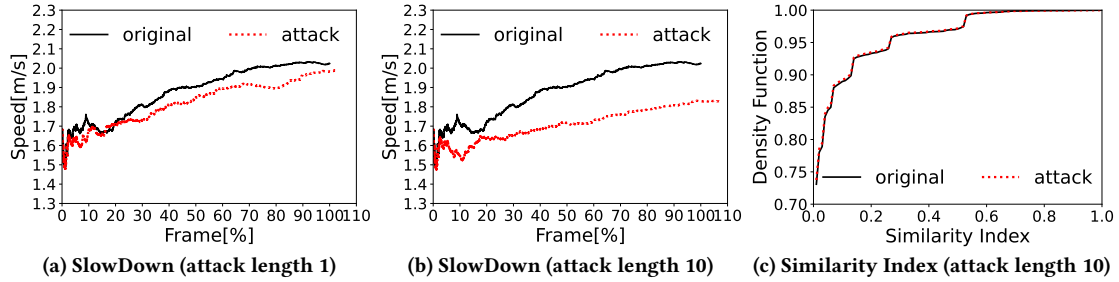


Figure 7: SlowDown Attack. The magnitude of SlowDown depends on attack length (small at 1 (a) vs large at 10 (b)). (c) shows that the distribution of hashing function which does not change before and after the attack, demonstrating stealthiness.

10 frames attacked at once. It is worth noting that when attack length is smaller, the speed fluctuates quite a lot in the attacked sequence. This is due to the fact that attacker kept attacking but the attacks were small enough and the tracking algorithm is able to correct some of the unexpected deviations. On the other hand, when attacks are large and happen more often, the reality of the tracking algorithm is changed and it cannot distinguish between an attack and normal path.

Figure 6a and 6b demonstrate the effectiveness of our attack approach in launching a configurable attack. Figure 6c demonstrates the stealthiness of the launched attack. By dropping the frames, the attacked was able to achieve a speedup of around $0.3m/s$. However, it did not significantly change the distribution of the similarity index across the overall timespan. A sanity check mechanism that looks for the change in similarity index across the frames would not be able to detect the attack.

As mentioned above, the final effect of the *SpeedUp* attack depends on how the user is using the application. In addition to the aforementioned slowdown effect, this could have other implications for the user. For example, if the goal of the application is to estimate the user trajectory and guide the user on a specific path, it may deviate the user from the actual path. This will happen as the trajectory algorithms such as SelectFusion that depend on history will have in correct history due to dropped frames.

6.2.2 SlowDown Attack. In launching a *SlowDown* attack, the goal of the attacker is to give the application an impression that the user is walking at a slower speed than its actual speed. Just like the *SpeedUp* attack, the end effect on the user will depend on the goals set for the application by the user. Lets reconsider the previous example of a user with an objective of walking at a desired fixed speed. The expected result from the attacker is that the application will generate a feedback for the user to walk faster, leading to an actual speedup effect.

Figure 7 shows the effect of *SlowDown* attack under varying attack parameters and its effect on the similarity index distribution. To reiterate, an *SlowDown* attack is achieved by holding the frames causing the scene to change slowly, leading to a perception of user moving slower. Figure 7a shows that the slowdown after the attack is small. However, the attacked trajectory has more number of frames the attack holds the frames and resumes the normal sequence of frames once the attack ceases. The change in speed and the difference in the number of frames is more visible in Figure 7b, where the attacker launches a more severe attack and holds a single frame for 10 frames. Similar to the *SpeedUp* attack, when attack length is smaller, the speed fluctuates quite a lot in the attacked sequence while remaining stable when the attack lengths are longer.

Figure 7a and 7b demonstrate the effectiveness of our attack approach in launching a configurable attack. Figure 7c demonstrates the stealthiness of the launched attack. Just by holding the frames, the attacked was able to achieve a slowdown of around $0.3m/s$. However, it did not significantly change the distribution of the similarity index across the overall timespan. A sanity check mechanism that looks for the change in similarity index across the frames would not be able to detect the attack.

As mentioned above, the final effect of the *SlowDown* attack depends on how the user is using the application, similar to the *SpeedUp* attack. In addition to the aforementioned speedup effect, this could have other implications for the user. The application would guide the user to walk faster, which could be physically harmful for the user as well as the people surrounding the user.

6.2.3 Zero Displacement Deviation Attack. In launching a *Zero Displacement* attack, the goal of the attacker is to give the application an impression that the user is periodically changing its speed, walking faster at times whole slowing down at others. Since this attack is launched using a combination of *SpeedUp* and *SlowDown* attack, the end effect depends on the when each of the attacks is

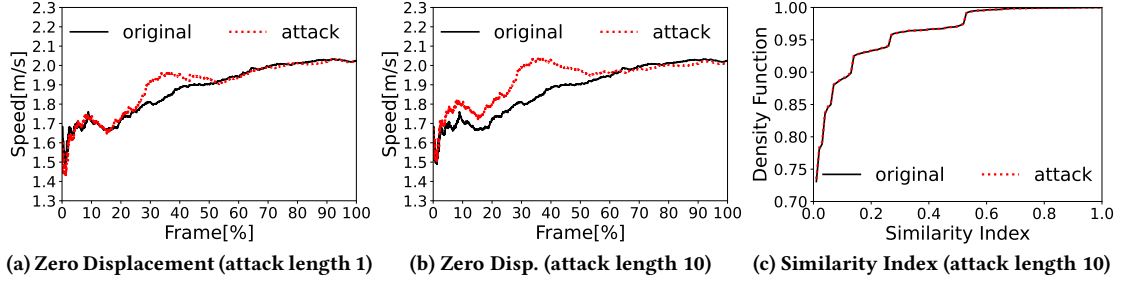


Figure 8: Zero Displacement Attack. The magnitude of deviation depends on attack length (small at 1 (a) vs large at 10 (b)). (c) shows that the distribution of hashing function which does not change before and after the attack, demonstrating stealthiness.

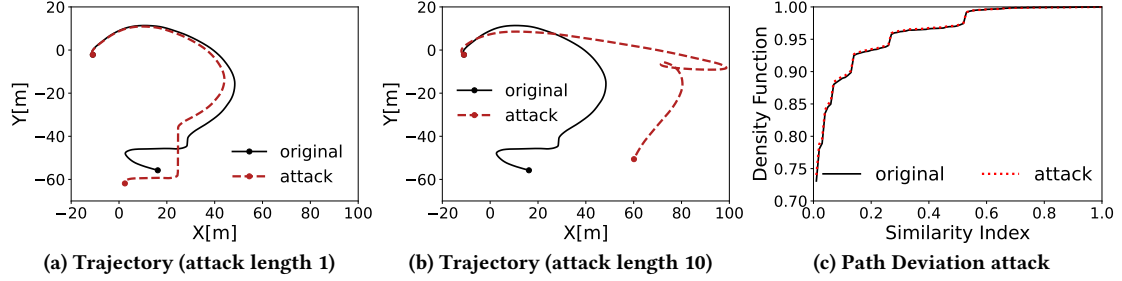


Figure 9: Path Deviation Attack. The trajectory is shown with attack length of 1 (a) vs attack length 10 (b). (c) shows that the distribution of hashing function as a similarity index does not change before and after the attack, demonstrating stealthiness.

launched and if one type of the attack dominates the other. If both attacks are launched alternatively for the same duration, the net effect would be zero. The user would have just followed a different path to the original trajectory.

Figure 8 demonstrates the effect of our zero displacement attack under various attack lengths. Figure 8a and 8b show that at start, speed is increasing slowly, which is followed by a big divergence, and finally speed starts to slow down then at the end matched the original sequence. In practical scenario attacker would not be able to know in advance speed of the user, so constantly shuffling of frames can get the attacker to the final destination at the same time. However if this attack is handled only with temporal knob the attacker might have diverted from the original destination even with shuffling of the frames, to bring it back to the final destination attacker needed to manipulate frames spatially to get back to final destination. Similar to the previous two attacks, Figure 8c shows that there was no statistically significant difference between the similarity index of the original and attacked sequences.

6.2.4 Path Deviation Attack. The *SpeedUp*, *SlowDown*, and *Zero Displacement* attacks only used temporal knobs for the attack. Our next attack, the *Path Deviation* attack, demonstrates the effect of manipulation via spatial knobs on both the attack vector, i.e. inertial and visual. In this attack, the attacker wants to change the direction of user and does not care about deviating from final destination.

Figure 9 shows the effect of *Path Deviation* attack under varying attack parameters and its effect on the similarity index distribution. Figure 9a shows that the trajectory after the orientation is shifted by only a small amount. The change in the orientation and the trajectory is more visible in Figure 9b, where the attacker launches a more severe attack and misaligns the frames by a bigger margin. Note that this is the long lasting effect of smaller changes throughout, the attacker did not change much but kept changing for longer duration and slowly the effect accumulated at the end. Overall the

attacker achieved an 32 meters change in position and 1.5 radians change in the orientation. Finally, as in previous attacks, Figure 9c shows no significant difference between the similarity index of original and attacked sequences.

6.3 Effect of Steadiness on Attacks

In launching the various aforementioned attacks, we outlined that the attack is launched during the steady-state. A steady-state is defined as having *num_steady_state* frames within a certain bound. However, not all steady-states are same and may have significant variations in the frames within the steady-state window depending on the context. For example, if a scene is changing, the standard deviation would be low and vice versa. We quantify this variation using standard deviation calculated over a given steady-state window and compare this value against the deviation for that window.

In this section we evaluate how the steady effect will affect the deviation. We compute the standard deviation among similarity index on the last frame of the frame window that constituted the "steady period". In figure 10 we can see that across all the attacks most of our attacks are launched across frame window with lowest standard deviation between the similarity index. But it can also be seen that some of the attacks are launched with high difference as well. This case is understandable if the scene is changing and algorithm is looking for attack frames and since the history is limited with attacker the attacker looks within the history if the similarity difference is decreasing or not. In that limited history that difference falls in to percentile range defined by the attacker.

We can also see that deviation w.r.t difference between similarity. In most cases the low standard deviation or a extremely steady-state results high deviation. This guarantees the stealthiness and also causes divergence that is intended by attacker. This result shows a significant potential of our proposed attack surface and algorithm. XR is becoming common in critical applications like surgery [65]

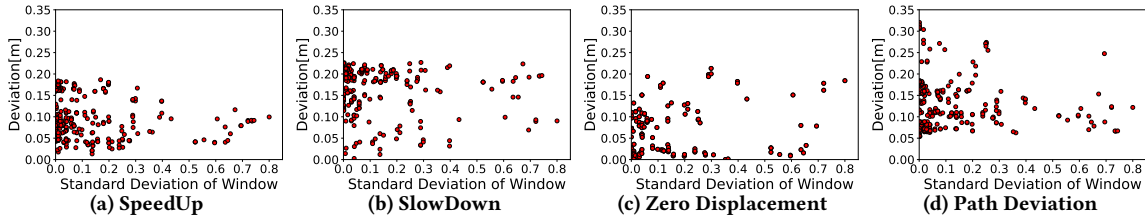


Figure 10: Effect of steadiness of a given window on the deviation achieved in that particular window. Low variation in a given window results in a more controllable and effective attack.

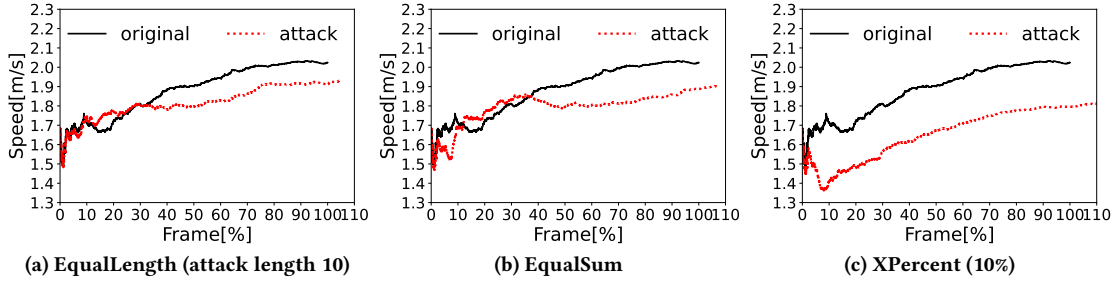


Figure 11: Difference between the speed on attacked sequence and original sequence.

Attack	Attack percentage	ks test statistic	Mean difference
SlowDown	7.2	0.06	0.14
SpeedUp	7.2	0.07	0.11
Zero Displacement	7.2	0.05	0.03
Path Deviation	7.2	0.07	0.15
FixedLength	4.5	0.07	0.23
EqualSum	7.2	0.08	0.35
XPercent	10	0.13	0.49

Table 2: Comparison of different frame picking schemes

and in those applications micro movements are the ones that are tracked. More applications are rehabilitation also have the micro movement where the physical scene does not change much but movements are critical. Our attack surface takes steadiness as essential mode so our attack will also be effective in the scenarios where the micro movements are essential to capture, and attacking that can cause serious consequence. Even if the scene is not changing the deviation is still attainable in those scenarios.

6.4 Robustness of RFRS algorithm

In the prior section, we demonstrated the efficacy and the stealthiness of different attack types that leverage our RFRS algorithm. We also demonstrated how different levels of steadiness impact the configurability and effectiveness of the attacks. We next demonstrate that a frame picking scheme that is agnostic of the steadiness is less effective, less configurable, and less stealthy.

To demonstrate this, we use a steadiness-agnostic periodic frame-picking scheme with three different configurations. The three configurations are (1) *FixedLength*, where the attacker launches the attack periodically after every N frames for a fixed attack length, e.g. 10 frames, (2) *EqualSum*, this schemes launches the attacks periodically but makes sure that the number of attacked frames is same as ours RFRS algorithm. (3) *XPercent*: in this scheme we attacked $X\%$ frames of the sequence.

Figures 11 demonstrates the various configuration of the periodic frame-picking scheme to launch a *SlowDown* attack. In Figure 11a, the *FixedLength* frame-picking scheme launches the attack. However, the attacker is not able to launch a consistent *SlowDown* attack, as opposed to a successful attack using RFRS algorithm (Figure 7b).

The attack results in even a speedup during some segments of the sequence. It is worth noting that the total number of frames attacked in the *FixedLength* attack is not same as RFRS algorithm with attack length of 10. To make a fair comparison, we used the *EqualSum* configuration where the total number of attacked frames is same in both cases. Figure 11b shows the *EqualSum* frame-picking scheme. While we see an improvement in the deviation, the attacker is still not able to launch a precise *SlowDown* attack.

The final configuration of the attack is shown in Figure 11c. While this attack seems to be highly effective, the graph does not paint a complete picture. Table 2 presents a comparison of all the attacks we have launched so far based on various metrics. We can see that the ks test value and the mean difference for the *XPercent* attack is the highest, which means that this attack is the least stealthy attack in our set. This will result in the attack being detected by the system, as it has changed the distribution of the similarity index for the sequence significantly after the attack.

6.5 Context-awareness of Attacks

In our evaluations so far, we picked an attack type, e.g. *SlowDown*, and launched the same attack for the whole duration. However, this does not show the full strength of our approach. Since our attack mechanism makes local decisions and does not assume anything about the future, it allow the attacker to pick different attacks based on the context. This can be helpful in situations when the scene is changing and the attacker is better served picking from a range of attacks based on the situation and the intended result. One example of this multiple attack scenario is where the person is performing exercises one after the other while getting direction from the Extended Reality (XR) headset. In this scenario the context will keep shifting so the type of attack needs to change as well.

Figure 12 illustrates different scenarios and the type of attack we launched based on the scenario. In this sequence, the user was first jogging and the attacker goal is to increase the jogging pace of the user to a dangerous level. As discussed before, this can be achieved by launching a *SlowDown* attack where the application gets an impression that the user has slowed down and will ask

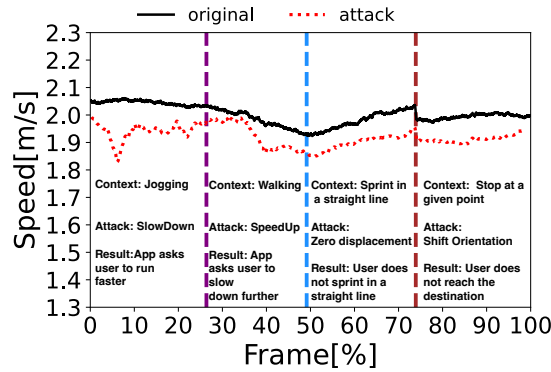


Figure 12: Our attacks rely only on local information and allow attacker to change attacks based on the context.

the user to speed up. After a patch of jogging, our user shifted to walking as is common with many jogging routines. During this phase, the user would like to maintain a fast walking speed to not completely cool down the body, making the subsequent runs harder. At this point, the attacker launches the *SpeedUp* attack giving the application an impression that user is walking too fast, almost same as jogging. The intended result is that the application will ask the user to slowdown resulting in cooling down the body which will make the subsequent runs harder.

The next phase in the user's jogging routine was a short sprint. The user wants to run in a straight line to reach a high speed. However, the attacker launches *Zero Displacement* attack to deviates the user from a straight trajectory making the sprint dangerous and less-effective. Finally, the user switches to jogging and want to stop at a particular destination. The attacker at this point wants to deviate the user from reaching the final destination and launches a *Path Deviation* attack. The user ends up at a different destination.

6.6 Microbenchmarks

We present benchmarking results to demonstrate that our attacks are fast and simple, which allows them to be used in a fast-paced environment of XR. For a successful and stealthy launch, the attacker needs to decide on launching the attack, deciding the attack type, compute similarity index, and perform frame manipulations. In our use case, images have 5-30 fps rate, while inertial sensors have 12-20 fps. As a result, our attack should not take more than 33.33ms (this calculated for the attack vector with the highest sampling rate). We empirically calculated the time it takes to complete all of the different attack steps. The calculation of similarity index for the inertial vector takes on average 0.8ms (1.5ms max and 0.7ms min). For visual attack vector, it takes on average 10ms (13.8ms max and 9.2ms min). The frame manipulation step takes 5ms on average. Finally, the RFRS algorithm runs in 0.0011 ms. The total time is less than 24ms on average, which is less than the highest sampling rate making our attacks fast and stealthy.

7 RELATED WORK

Extended Reality (XR) Security. Prior research on XR security focuses on vulnerability analysis in applications interactions [9, 16, 19, 32], secure authentication and pairing [18, 55], security across users [28, 33, 48, 71], and privacy concerns in remote collaborations [7, 46, 57]. No prior work explores the multi-modal and multi-dimensional attack surface presented here.

Spatiotemporal Sensor Attacks. There is a lot of prior work on sensor attacks across multiple dimensions. Data-level temporal and spatial attacks have been demonstrated for the inertial sensors [59, 60] as well as visual sensors [62] using a range of manipulation strategies. Device-level manipulation of timing and location services has been shown to be effective as well [5, 6]. Prior work has also demonstrated spatiotemporal attacks that lead to misalignment of content in XR and inaccurate tracking [45, 52]. However, these approaches ignore the concurrent launch of attacks across multiple attack vectors that is required to overcome the sensor fusion algorithms and systems checks for stealthy operation.

Sensor Fusion. The state-of-the-art sensor fusion algorithms leverage deep-learning based tracking on top of visual inertial odometry (VIO), as used by Hololens 2 [24]. Chen et al. [11] present robust deep learning-based multi-modal fusion for VIO by relying on the latent features from different sensor modalities, particularly under degraded environmental conditions such as noise, bias, occlusions, spatiotemporal misalignment, and missing data. In contrast, a malicious spatial and temporal misalignment would distort the fundamental characteristics of latent features, thus presenting new challenges in secure sensor fusion, and motivates further exploration on spatiotemporal attacks which is the focus of our work.

8 DISCUSSION

We discuss possible remediation techniques for our proposed attack surface and how they remain ineffective in detecting and mitigating the attacks. Additionally, we discuss how the proposed attacks are not only applicable to Extended Reality (XR) but also apply to other relevant domains with similar spatiotemporal semantics.

Potential Defense Mechanisms. Previous works have proposed data production level defenses such as acoustic dampening [10] that helps reduce resonant interference in the inertial signal. Better filtering approaches are proposed to mitigate the spoofing on inertial sensors [59]. Visual challenges have been used to detect camera attacks [63]. Other relevant countermeasure technique is to add watermark in the images to protect the frame [41]. We argue that these remediation strategies require either a fundamental shift in sensor hardware design or heavily instrumenting the environment, which are not feasible in current deployments. Complex deep neural networks are also ineffective in neutralizing concurrent manipulations across the sensors. In fact, these algorithms start adapting to malicious inputs induced by the attacker.

Broad Applicability. Our approach has broader impacts both at different tracking scales and also across different domains. For instance, our tracking use cases in navigation and exercising have movements that stretch to few inches to meters. Our attack is also applicable to surgical use case with micro-level movements. Our attack surface is transferable to other tracking-based systems such as autonomous vehicles [12, 36], smart robots [68], and drones [15].

9 CONCLUSION

We present a new attack surface and demonstrate its validity in XR systems. The success of these attacks depends on concurrent and context-aware manipulations of multiple sensing modalities across spatiotemporal dimensions to achieve the desired outcome in a stealthy manner. Future research avenue of this work is micro-level manipulations in eye tracking applications.

REFERENCES

- [1] 2008. *Kolmogorov–Smirnov Test*. Springer New York, New York, NY, 283–287. https://doi.org/10.1007/978-0-387-32833-1_214
- [2] Jad Al Rabbah, Alexis Morris, and Sowmya Somanath. 2019. MRsive: An Augmented Reality Tool for Enhancing Wayfinding and Engagement with Art in Museums. In *HCI International 2019 - Posters*, Constantine Stephanidis (Ed.). Springer International Publishing, Cham, 535–542.
- [3] Paul G. Allen. 2021. Security and Privacy for Augmented Reality: Our 10-Year Retrospective.
- [4] Sean Andrist, Dan Bohus, Ashley Feniello, and Nick Saw. 2022. Developing Mixed Reality Applications with Platform for Situated Intelligence. In *2022 IEEE Conference on Virtual Reality and 3D User Interfaces Abstracts and Workshops (VRW)*. 48–50. <https://doi.org/10.1109/VRW55335.2022.00018>
- [5] F.M. Anwar, L. Garcia, X. Han, and M. Srivastava. 2019. Securing Time in Untrusted Operating Systems with TimeSeal. In *2019 IEEE RTSS*.
- [6] F.M. Anwar and M. Srivastava. 2020. A Case for Feedforward Control with Feedback Trim to Mitigate Time Transfer Attacks. *ACM TOPS* (2020).
- [7] Huidong Bai, Prasanth Sasikumar, Jing Yang, and Mark Billinghurst. 2020. A User Study on Mixed Reality Remote Collaboration with Eye Gaze and Hand Gesture Sharing. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) (CHI '20). Association for Computing Machinery, New York, NY, USA, 1–13. <https://doi.org/10.1145/3313831.3376550>
- [8] Yulong Cao, Chaowei Xiao, Benjamin Cyr, Yimeng Zhou, Won Park, Sara Rampazzi, Qi Alfred Chen, Kevin Fu, and Z. Morley Mao. 2019. Adversarial Sensor Attack on LiDAR-Based Perception in Autonomous Driving. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security* (London, United Kingdom) (CCS '19). Association for Computing Machinery, New York, NY, USA, 2267–2281. <https://doi.org/10.1145/3319535.3339815>
- [9] P. Casey, I. Baggili, and A. Yarramreddy. 2019. Immersive Virtual Reality Attacks and the Human Joystick. *IEEE TDSC* (2019).
- [10] Sneider Castro, Robert Dean, Grant Roth, George Flowers, and Brian Grantham. 2007. Influence of Acoustic Noise on the Dynamic Performance of MEMS Gyroscopes. <https://doi.org/10.1115/IMECE2007-42108>
- [11] C. Chen, S. Rosa, C.X. Lu, N. Trigoni, and A. Markham. 2019. Select-Fusion: A Generic Framework to Selectively Learn Multisensory Fusion. arXiv:1912.13077 [cs.CV]
- [12] Duanfeng Chu, Haoran Li, Chenyang Zhao, and Tuqiang Zhou. 2022. Trajectory Tracking of Autonomous Vehicle Based on Model Predictive Control With PID Feedback. *IEEE Transactions on Intelligent Transportation Systems* (2022), 1–12. <https://doi.org/10.1109/TITS.2022.3150365>
- [13] Ronald Clark, Sen Wang, Hongkai Wen, A. Markham, and Agathoniki Trigoni. 2017. ViNet: Visual-Inertial Odometry as a Sequence-to-Sequence Learning Problem. In *AAAI*.
- [14] Angela Dai, Matthias Nießner, Michael Zollhöfer, Shahram Izadi, and Christian Theobalt. 2017. BundleFusion: Real-Time Globally Consistent 3D Reconstruction Using On-the-Fly Surface Reintegration. *ACM Trans. Graph.* 36, 3, Article 24 (may 2017), 18 pages. <https://doi.org/10.1145/3054739>
- [15] Louis Dressel and Mykel J. Kochenderfer. 2019. Hunting Drones with Other Drones: Tracking a Moving Radio Target. In *2019 International Conference on Robotics and Automation (ICRA)*. 1905–1912. <https://doi.org/10.1109/ICRA.2019.8794243>
- [16] Reyhan Duezguen, Peter Mayer, Sanchari Das, and Melanie Volkamer. 2020. Towards Secure and Usable Authentication for Augmented and Virtual Reality Head-Mounted Displays. *CoRR abs/2007.11663* (2020). arXiv:2007.11663 <https://arxiv.org/abs/2007.11663>
- [17] Khaled El Ammari and Amin Hammad. 2019. Remote interactive collaboration in facilities management using BIM-based mixed reality. *Automation in Construction* 107 (2019), 102940. <https://doi.org/10.1016/j.autcon.2019.102940>
- [18] Ethan Gaebel, Ning Zhang, Wenjing Lou, and Y. Thomas Hou. 2016. Looks Good To Me: Authentication for Augmented Reality. In *Proceedings of the 6th International Workshop on Trustworthy Embedded Devices* (Vienna, Austria) (TrustED '16). Association for Computing Machinery, New York, NY, USA, 57–67. <https://doi.org/10.1145/2995289.2995295>
- [19] C. George, M. Khamis, D. Buschek, and H. Hussmann. 2019. Investigating the Third Dimension for Authentication in Immersive Virtual Reality and in the Real World. In *2019 IEEE Conference on Virtual Reality and 3D User Interfaces (VR)*. 277–285.
- [20] Águeda Gómez-Cambronero, Alberto González-Pérez, Ignacio Miralles, and Sven Casteleyn. 2019. MIXED REALITY ESCAPE ROOM VIDEO GAME TO PROMOTE LEARNING IN INDOOR ENVIRONMENTS. *EDULEARN19 Proceedings* (2019).
- [21] Qingying Hao, Licheng Luo, Steve T.K. Jan, and Gang Wang. 2021. It's Not What It Looks Like: Manipulating Perceptual Hashing Based Applications. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security* (Virtual Event, Republic of Korea) (CCS '21). Association for Computing Machinery, New York, NY, USA, 69–85. <https://doi.org/10.1145/3460120.3484559>
- [22] Jassim Happa, Mashhuda Glencross, and Anthony Steed. 2019. Cyber Security Threats and Challenges in Collaborative Mixed-Reality. *Frontiers in ICT* 6 (2019). <https://doi.org/10.3389/fict.2019.00005>
- [23] Zhitao He and Thiemo Voigt. 2013. Droplet: A New Denial-of-Service Attack on Low Power Wireless Sensor Networks. In *2013 IEEE 10th International Conference on Mobile Ad-Hoc and Sensor Systems*. 542–550. <https://doi.org/10.1109/MASS.2013.18>
- [24] Microsoft Inc. 2020. Hololens 2. <https://www.microsoft.com/en-us/hololens/hardware>. (2020). "[Online; accessed 13th-June-2022]".
- [25] Omid Jafari, Preeti Maurya, Parth Nagarkar, Khandker Mushfiqul Islam, and Chidambaram Crushev. 2021. A Survey on Locality Sensitive Hashing Algorithms and their Applications. <https://doi.org/10.48550/ARXIV.2102.08942>
- [26] Ahmad Jalal, Mouazma Batool, and Kibum Kim. 2020. Sustainable Wearable System: Human Behavior Modeling for Life-Logging Activities Using K-Ary Tree Hashing Classifier. *Sustainability* 12, 24 (2020). <https://doi.org/10.3390/su122410324>
- [27] Nicolas Kakouris, Numan Yener, and Daniel T.P. Fong. 2021. A systematic review of running-related musculoskeletal injuries in runners. *Journal of Sport and Health Science* 10, 5 (2021), 513–522. <https://doi.org/10.1016/j.jshs.2021.04.001>
- [28] Mohamed Khamis, Karola Marky, Andreas Bulling, and Florian Alt. 2022. User-centred multimodal authentication: securing handheld mobile devices using gaze and touch input. *Behaviour & Information Technology* 0, 0 (2022), 1–23. <https://doi.org/10.1080/0144929X.2022.2069597> arXiv:https://doi.org/10.1080/0144929X.2022.2069597
- [29] Rachel C. King, Emma Villeneuve, Ruth J. White, R. Simon Sherratt, William Holderbaum, and William S. Harwin. 2017. Application of data fusion techniques and technologies for wearable health monitoring. *Medical Engineering & Physics* 42 (2017), 1–12. <https://doi.org/10.1016/j.medengphy.2016.12.011>
- [30] Anunay Kulshrestha. 2012. On the Hamming Distance between base-n representations of whole numbers. <https://doi.org/10.48550/ARXIV.1203.4547>
- [31] Kiron Lebeck, Tadayoshi Kohno, and Franziska Roesner. 2016. How to Safely Augment Reality: Challenges and Directions. In *Proceedings of the 17th International Workshop on Mobile Computing Systems and Applications* (St. Augustine, Florida, USA) (HotMobile '16). Association for Computing Machinery, New York, NY, USA, 45–50. <https://doi.org/10.1145/2873587.2873595>
- [32] Kiron Lebeck, Tadayoshi Kohno, and Franziska Roesner. 2019. Enabling Multiple Applications to Simultaneously Augment Reality: Challenges and Directions. In *Proceedings of the 20th International Workshop on Mobile Computing Systems and Applications* (Santa Cruz, CA, USA) (HotMobile '19). Association for Computing Machinery, New York, NY, USA, 81–86. <https://doi.org/10.1145/3301293.3302362>
- [33] K. Lebeck, K. Ruth, T. Kohno, and F. Roesner. 2017. Securing Augmented Reality Output. In *2017 IEEE Symposium on Security and Privacy (SP)*. 320–337.
- [34] Mingyang Li and Anastasios Mourikis. 2013. High-precision, consistent EKF-based visual-inertial odometry. *The International Journal of Robotics Research* 32 (05 2013), 690–711. <https://doi.org/10.1177/0278364913481251>
- [35] Yang Li, Jianke Zhu, Steven C. H. Hoi, Wenjie Song, Zhefeng Wang, and Hantang Liu. 2017. Robust Estimation of Similarity Transformation for Visual Object Tracking. <https://doi.org/10.48550/ARXIV.1712.05231>
- [36] Fen Lin, Mingbiao Hao, Minghong Sun, Yuke Chen, Jian Wu, and Chengliang Qian. 0. Path tracking of autonomous vehicle with consideration of vehicle stability in sideslip angle phase plane. *Proceedings of the Institution of Mechanical Engineers, Part D: Journal of Automobile Engineering* 0, 0 (0), 09544070221083728. <https://doi.org/10.1177/09544070221083728> arXiv:https://doi.org/10.1177/09544070221083728
- [37] Yonggen Ling, Linchao Bao, Zequn Jie, Fengming Zhu, Ziyang Li, Shanmin Tang, Yongsheng Liu, Wei Liu, and Tong Zhang. 2018. Modeling Varying Camera-IMU Time Offset in Optimization-Based Visual-Inertial Odometry. <https://doi.org/10.48550/ARXIV.1810.05456>
- [38] Fangfang Lu, Hao Zhou, Lingling Guo, Jingjing Chen, and Licheng Pei. 2021. An ARCore-Based Augmented Reality Campus Navigation System. *Applied Sciences* 11, 16 (2021). <https://doi.org/10.3390/app11167515>
- [39] Arif Mahmood and Sohaib Khan. 2007. Exploiting Inter-Frame Correlation for Fast Video to Reference Image Alignment. In *Proceedings of the 8th Asian Conference on Computer Vision - Volume Part I* (Tokyo, Japan) (ACCV'07). Springer-Verlag, Berlin, Heidelberg, 647–656.
- [40] Elmar Mair, Michael Fleps, Michael Suppa, and Darius Burschka. 2011. Spatio-temporal initialization for IMU to camera registration. In *2011 IEEE International Conference on Robotics and Biomimetics*. 557–564. <https://doi.org/10.1109/ROBIO.2011.6181345>
- [41] M Murty, D Veeraiah, and A Rao. 2011. Digital Signature and Watermark Methods For Image Authentication using Cryptography Analysis. *An International Journal (SIPIJ)* 2 (07 2011). <https://doi.org/10.5121/sipij.2011.2214>
- [42] Deeraj Nagothu, Yu Chen, Erik Blasch, Alexander Aved, and Sencun Zhu. 2019. Detecting Malicious False Frame Injection Attacks on Surveillance Systems at the Edge Using Electrical Network Frequency Signals. *Sensors* 19, 11 (2019). <https://doi.org/10.3390/s19112424>
- [43] Keith Nesbitt, Simon Davis, Karen Blackmore, and Eugene Nalivaiko. 2017. Correlating reaction time and nausea measures with traditional measures of cybersickness. *Displays* 48 (2017), 1–8. <https://doi.org/10.1016/j.displa.2017.01.002>

- [44] Daniel Pimentel and Charlotte Vinkers. 2021. Copresence With Virtual Humans in Mixed Reality: The Impact of Contextual Responsiveness on Social Perceptions. *Frontiers in Robotics and AI* 8 (04 2021). <https://doi.org/10.3389/frobt.2021.634520>
- [45] Samina Rafique, M Najam ul Islam, M Shafique, and A Mahmood. 2020. Neuro-fuzzy control of sit-to-stand motion using head position tracking. *Measurement and Control* 53, 7-8 (2020), 1342–1353. <https://doi.org/10.1177/0020294020938079> arXiv:<https://doi.org/10.1177/0020294020938079>
- [46] Xukan Ran, Carter Slocum, Maria Gorlatova, and Jiasi Chen. 2019. ShareAR: Communication-Efficient Multi-User Mobile Augmented Reality. In *Proceedings of the 18th ACM Workshop on Hot Topics in Networks* (Princeton, NJ, USA) (*HotNets '19*). Association for Computing Machinery, New York, NY, USA, 109–116. <https://doi.org/10.1145/3365609.3365867>
- [47] René Reinhard, Ender Tutulmaz, Hans Maria Rutrecht, Patricia Hengstenberg, Britta Geissler, Heiko Hecht, and Axel Mutttray. 2019. Effects of Visually Induced Motion Sickness on Emergency Braking Reaction Times in a Driving Simulator. *Human Factors: The Journal of Human Factors and Ergonomics Society* 61 (2019), 1004 – 1018.
- [48] Kimberly Ruth, Tadayoshi Kohno, and Franziska Roesner. 2019. Secure Multi-User Content Sharing for Augmented Reality Applications. In *USENIX Security Symposium*.
- [49] Priyanka Samanta and Shweta Jain. 2021. Analysis of Perceptual Hashing Algorithms in Image Manipulation Detection. *Procedia Computer Science* 185 (2021), 203–212. <https://doi.org/10.1016/j.procs.2021.05.021> Big Data, IoT, and AI for a Smarter Future.
- [50] Thomas Schöps, Torsten Sattler, and Marc Pollefeys. 2019. BAD SLAM: Bundle Adjusted Direct RGB-D SLAM. In *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 134–144. <https://doi.org/10.1109/CVPR.2019.00022>
- [51] Mahmood Sharif, Sruti Bhagavatula, Lujo Bauer, and Michael K. Reiter. 2016. Accessorize to a Crime: Real and Stealthy Attacks on State-of-the-Art Face Recognition. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security* (Vienna, Austria) (*CCS '16*). Association for Computing Machinery, New York, NY, USA, 1528–1540. <https://doi.org/10.1145/2976749.2978392>
- [52] R. A. Sharma, A. Dongare, J. Miller, N. Wilkerson, D. Cohen, V. Sekar, P. Dutta, and A. Rowe. 2020. All that GLITTERs: Low-Power Spoof-Resilient Optical Markers for Augmented Reality. In *2020 19th ACM/IEEE International Conference on Information Processing in Sensor Networks (IPSN)*. 289–300.
- [53] Amit Kumar Sikder, Hidayet Aksu, and A. Selcuk Uluagac. 2017. 6thSense: A Context-aware Sensor-based Attack Detector for Smart Devices. In *26th USENIX Security Symposium (USENIX Security 17)*. USENIX Association, Vancouver, BC, 397–414. <https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/sikder>
- [54] Vivek Singh, Pallav Pant, and Ramesh Tripathi. 2015. Detection of Frame Duplication Type of Forgery in Digital Video Using Sub-block Based Features. https://doi.org/10.1007/978-3-319-25512-5_3
- [55] I. Sluganovic, M. Serbec, A. Derek, and I. Martinovic. 2017. HoloPair: Securing Shared Augmented Reality Using Microsoft HoloLens. In *ACSAC*.
- [56] Shamus P. Smith and Elizabeth L. Burd. 2019. Response activation and inhibition after exposure to virtual reality. *Array* 3-4 (2019), 100010. <https://doi.org/10.1016/j.array.2019.100010>
- [57] Jovan Stojkovic, Zida Liu, Guohao Lan, Carlee Joe-Wong, and Maria Gorlatova. 2019. Edge-Assisted Collaborative Image Recognition for Augmented Reality: Demo Abstract. In *Proceedings of the 17th Conference on Embedded Networked Sensor Systems* (New York, New York) (*SenSys '19*). Association for Computing Machinery, New York, NY, USA, 394–395. <https://doi.org/10.1145/3356250.3361944>
- [58] Jennifer J. Sun, Jiaping Zhao, Liang-Chieh Chen, Florian Schroff, Hartwig Adam, and Ting Liu. 2019. View-Invariant Probabilistic Embedding for Human Pose. <https://doi.org/10.48550/ARXIV.1912.01001>
- [59] Timothy Trippel, Ofir Weisse, Wenyuan Xu, Peter Honeyman, and Kevin Fu. 2017. WALNUT: Waging Doubt on the Integrity of MEMS Accelerometers with Acoustic Injection Attacks. 3–18. <https://doi.org/10.1109/EuroSP.2017.42>
- [60] Y. Tu, Z. Lin, I. Lee, and X. Hei. 2018. Injected and Delivered: Fabricating Implicit Control over Actuation Systems by Spoofing Inertial Sensors. In *USENIX Security*.
- [61] Dorin Ungureanu, Federica Bogo, S. Galliani, Pooja Sama, Xin Duan, Casey Meekhof, Jan Stühmer, Thomas J. Cashman, Bugra Tekin, Johannes L. Schönberger, Pawel Olszta, and Marc Pollefeys. 2020. HoloLens 2 Research Mode as a Tool for Computer Vision Research. *ArXiv abs/2008.11239* (2020).
- [62] J. Valente, K. Bahirat, K. Venechanos, A.A. Cardenas, and P. Balakrishnan. 2019. Improving the Security of Visual Challenges. *ACM TCPS* (2019).
- [63] Junia Valente and Alvaro Cardenas. 2015. Using Visual Challenges to Verify the Integrity of Security Cameras. <https://doi.org/10.1145/2818000.2818045>
- [64] Mathy Vanhoef. 2021. Fragment and Forge: Breaking Wi-Fi Through Frame Aggregation and Fragmentation. In *Proceedings of the 30th USENIX Security Symposium*. USENIX Association.
- [65] Petr Vávra, Jan Roman, P. Zonča, Peter Ihnát, M. Němec, Kumar Jayant, Nagy Habib, and Ahmed El-Gendi. 2017. Recent Development of Augmented Reality in Surgery: A Review. *Journal of Healthcare Engineering* 2017 (08 2017), 1–9. <https://doi.org/10.1155/2017/4574172>
- [66] Carolin Wienrich, Philipp Komma, Stephanie Vogt, and Marc E. Latoschik. 2021. Spatial Presence in Mixed Realities—Considerations About the Concept, Measures, Design, and Experiments. *Frontiers in Virtual Reality* 2 (2021), 141. <https://doi.org/10.3389/frvir.2021.694315>
- [67] Yi Xu, Yuzhang Wu, and Hui Zhou. 2018. Multi-Scale Voxel Hashing and Efficient 3D Representation for Mobile Augmented Reality. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*.
- [68] Hui Zhang, Li Zhu Liu, He Xie, Yiming Jiang, Jian Zhou, and Yaonan Wang. 2022. Deep Learning-Based Robot Vision: High-End Tools for Smart Manufacturing. *IEEE Instrumentation & Measurement Magazine* 25, 2 (2022), 27–35. <https://doi.org/10.1109/MIM.2022.9756392>
- [69] Jianyu Zhao, Ergo Pikkas, Olli Seppänen, and Antti Peltokorpi. 2021. Using Real-Time Indoor Resource Positioning to Track the Progress of Tasks in Construction Sites. *Frontiers in Built Environment* 7 (2021). <https://doi.org/10.3389/fbuil.2021.661166>
- [70] Xiaoming Zhao, Jingmeng Liu, Xingming Wu, Weihai Chen, Fanghong Guo, and Zhengguo Li. 2022. Probabilistic Spatial Distribution Prior Based Attentional Keypoints Matching Network. *IEEE Transactions on Circuits and Systems for Video Technology* 32, 3 (2022), 1313–1327. <https://doi.org/10.1109/TCSVT.2021.3068761>
- [71] Fengyuan Zhu and Tovi Grossman. 2020. BISHARE: Exploring Bidirectional Interactions Between Smartphones and Head-Mounted Augmented Reality. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) (*CHI '20*). Association for Computing Machinery, New York, NY, USA, 1–14. <https://doi.org/10.1145/3313831.3376233>
- [72] Wenhao Zhu, Tengjun Yao, Jianyue Ni, Baogang Wei, and Zhiguo Lu. 2018. Dependency-based Siamese long short-term memory network for learning sentence representations. *PLOS ONE* 13, 3 (03 2018), 1–14. <https://doi.org/10.1371/journal.pone.0193919>